SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO1: Analyze virtualization architectures to identify performance and security issues in cloud computing environments..

Field	Details	Field	Details
Student Name	C. Jayalakshmi	Register Number	192524052
Department / Section	AI&DS	Academic Year / Semester	2026
Faculty Name	Dr. D. Antony Joseph Raja	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	6/12/2026	Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Introduce Cloud Computing	3		Definition; characteristics; deployment models; scenario mapping
2	Trace Evolution and Hardware Evolution	3		Timeline; hardware evolution; relevance to university
3	Analyse Internet Software Evolution	3		Web/software evolution; SOA; APIs; scenario relevance

4	Apply Server Virtualization	5		Hypervisor types; VM/container comparison; consolidation; limitations
5	Apply Web Services	4		SOA and SOAP/REST; three university APIs; request/response examples
6	Analyse IaaS, PaaS and SaaS	5		Responsibility layers; five-application mapping; comparison; providers
7	Examine XaaS	3		Four or more variants; two justified choices; provider risks
8	Design Cloud Solution Architecture	5		Labelled diagram; deployment/service models; scaling; security; DR
9	Hands-on Implementation and Documentation	4		Running VM/container; PaaS/SaaS; tests; actual screenshots
10	Evaluate and Recommend Final Solution	7		Three-way comparison; 3-year costs; risks; phased roadmap
11	Day-4 Design Challenge	8		Impact; evaluation; adaptation; evidence-based defence
	TOTAL	**50**		**Percentage :**

EVIDENCE VERIFICATION CHECKLIST

Tasks	**Evidence expected**	**Status (✓ / P / ✗)**	**Evaluator note**
1–3	Conceptual explanations, comparisons and relevant illustrations	✓	Explanations and relevant scenario mapping documented

4	Virtualization calculations with stated assumptions and peak-load caution	✓	Calculations present; practical capacity validation recommended
5–7	API examples, service responsibility table and justified XaaS choices	✓	Required conceptual evidence documented
8	Labelled architecture diagram, caption below figure and component justification	✓	Architecture documented; sizing assumptions need testing
9	Actual code, environment details, execution logs, API outputs and screenshots	P	Procedures provided; actual execution evidence not established
10	Cost assumptions, 3-year totals, risk controls and migration roadmap	✓	Estimates and rationale documented
11	Day-4 changes, 10× load strategy, 99.9% target and 40-college isolation	✓	Design analysis documented; practical validation pending

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	**Observation**
Source code	✓	Flask REST API and Dockerfile provided in the proposed implementation
Virtualization	P	Docker container configuration documented; running-container evidence required
PaaS or SaaS	P	Managed web-service deployment procedure described; live deployment not verified
Functional test	P	Expected HTTP 200 response and JSON output documented; actual output required
Failure case	P	HTTP 404 test case documented; actual execution record required

Screenshots	P	Screenshot requirements identified; authentic screenshots need verification
Reproducibility	✓	Build commands, run commands, endpoints and expected results specified

C .Jayalakshmi **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

Assignment Title:(Total Marks: 50)

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation
- How CPU, memory, storage and network are virtualized
- A brief justification for why this approach is relevant to EduCloud

ANSWER: I selected three different approaches: KVM full virtualisation, OS-level containers using Docker, and Kata Containers using lightweight virtual machines. They cover both VM-based and container-based virtualisation.

KVM – Type-1 / bare-metal virtualisation: Layered architecture: Physical hardware → Linux host with KVM kernel module and QEMU → separate guest operating system in each VM → guest runtime → SIS/LMS/application. KVM uses the processor's hardware virtualisation support (Intel VT-x or AMD-V).

KVM isolation and resources: Trust boundary: each VM is separated from other VMs by the KVM hypervisor and its virtual machine monitor. The host administrator and hypervisor are trusted. CPU time is scheduled as virtual CPUs; memory is mapped using extended/nested page tables; storage is presented through virtual disks and commonly VirtIO; networking uses virtual NICs connected through a virtual switch or bridge. This is suitable for SIS and LMS because different operating systems can run separately and the VM boundary is comparatively strong.

Docker – OS-level containers: Layered architecture: Physical hardware → Linux host operating system and kernel → container runtime (containerd/Docker) → isolated container namespaces and cgroups → application and its libraries. Containers share the host kernel and do not each run a separate guest kernel.

Docker isolation and resources: Trust boundary: the Linux kernel, namespaces, cgroups, capabilities and security modules enforce isolation. CPU is limited and scheduled using cgroups; memory is controlled by memory cgroups; storage is provided through layered filesystems and volumes; networking uses network namespaces, virtual Ethernet pairs and bridges. Containers start quickly and use less memory, so they are useful for stateless web services and scalable application components. However, sharing one kernel makes them less suitable as the only boundary between the university and untrusted external tenants.

Kata Containers – sandboxed container / micro-VM: Layered architecture: Physical hardware → host Linux and KVM → a lightweight VM for each pod/container → guest kernel → container runtime → application. The application is packaged like a container, but it runs inside a small virtual machine.

Kata isolation and resources: Trust boundary: the KVM hypervisor separates the guest kernel from the host and other guests; the guest kernel and container runtime provide a further boundary. CPU and memory are allocated to the micro-VM; virtual storage is commonly provided through VirtIO; networking uses virtual NICs and a virtual switch. Kata offers stronger isolation than ordinary containers while retaining a container workflow. It is useful for research workloads or tenants that should not share the host kernel with SIS.

Selection: These approaches are architecturally different: KVM provides full VMs, Docker provides shared-kernel containers, and Kata combines containers with VM isolation. They allow EduCloud to compare isolation, resource overhead and workload density fairly.

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended page-table walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

ANSWER: For KVM, the main overhead to examine is virtual I/O processing and VM exits. A guest operation that cannot be handled directly by hardware may trap to the hypervisor or QEMU. This occurs between the guest and the host virtual device. With VirtIO, the number of expensive exits is reduced, but processing still increases with disk and network activity. More busy VMs increase queueing and host CPU use.

Docker: For Docker, the main overhead is virtual networking and shared-kernel system-call processing. Packets pass through virtual Ethernet devices, bridges and firewall rules, while container system calls are handled by the common host kernel. More containers and higher packet rates increase context switches, packet processing and contention in shared kernel resources.

Kata Containers: For Kata, the main overhead is the extra VM boundary, including VM exits, guest-kernel work and virtual I/O. It occurs between the container workload, its guest kernel and the KVM host. It is important because every micro-VM adds memory and device-

processing work. As the number of micro-VMs or I/O rate increases, scheduling, memory use and virtual-device queues become more expensive.

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary
- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

ANSWER – assumptions: I assume one workload instance has 2 vCPUs and an average memory demand of 8 GB. The host reserves 32 GB for the hypervisor, host OS and

monitoring, leaving 480 GB for instances. One 25 Gbps NIC is treated as the network bottleneck. For illustration, the CPU virtualisation overhead allowance is 5% for KVM, 2% for Docker and 7% for Kata. These are modelling assumptions, not measured results; actual values must be measured on the test machine.

CPU model: Let r be the vCPU:pCPU overcommit ratio and h be the fraction of CPU time consumed by virtualisation overhead. The effective physical-core equivalent available to a 2-vCPU instance is $C = (2/r) \times (1 - h)$ cores, before additional workload contention. The four ratios give 2, 1, 0.5 and 0.25 core equivalents before overhead. At 2:1, the illustrative results are KVM: $1 \times 0.95 = 0.95$ cores; Docker: $1 \times 0.98 = 0.98$ cores; Kata: $1 \times 0.93 = 0.93$ cores. Steal time is the time a vCPU is ready but cannot run. Therefore, measured effective capacity is $C_{measured} = (2/r) \times (1 - h) \times (1 - s)$, where s is the measured steal-time fraction.

VM-exit cost: The given cost of one VM exit is 1.5 microseconds. If a workload causes E exits per second, the approximate CPU time spent handling exits per second is $E \times 1.5$ microseconds. The fraction of one CPU consumed is $E \times 1.5 \times 10^{-6}$. For example, 100,000 exits/second consume about 0.15 CPU-seconds per second, or 15% of one CPU core. This is an illustrative calculation; the actual exit rate must be collected.

Memory model: Usable memory = 512 − 32 = 480 GB. Maximum instances before memory overcommit = 480/8 = 60 instances. This is the memory ceiling for 8 GB per instance. In practice, the safe number should be lower if guest overhead, cache, memory spikes or host services need more memory. Docker containers may use less extra memory because they share the host kernel, while KVM and Kata need memory for guest kernels and VM processes.

I/O and network model: If the usable network rate is 90% of the 25 Gbps link, effective shared bandwidth is 22.5 Gbps. With N equally active instances, the simple fair-share estimate is $B = 22.5/N$ Gbps per instance. For 10 active instances, this is 2.25 Gbps each; for 20, it is 1.125 Gbps each. Real throughput depends on packet size, CPU processing, virtual-switch queues and traffic imbalance. For storage, if the measured usable device throughput is S, the equal-share estimate is S/N; the assignment does not provide an NVMe throughput value, so S must be measured rather than invented.

Density and service target: A practical density limit is the minimum of the memory limit, CPU limit and I/O limit: $N_{max} = \min(N_{memory}, N_{cpu}, N_{io})$. Memory gives 60 instances as an absolute upper bound. To keep performance degradation at or below 10%, reserve at least 10% of CPU capacity and avoid sustained swapping or saturated I/O. For a simple example,

if each instance needs 0.5 physical core at peak, 60 instances need 30 cores, which fits within 64 cores; however, the 120 assigned vCPUs create a 1.875:1 ratio. The service target must still be checked using measured steal time, latency and throughput.

Comparison: Docker normally has the lowest per-instance memory and CPU overhead. KVM has a higher fixed VM cost but a separate guest kernel. Kata usually costs more than ordinary containers because it adds a lightweight VM, but it gives a stronger boundary. The same equations apply to all three; the overhead fraction h and actual resource demand differ and must be obtained experimentally.

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

ANSWER – best case: KVM performs best when guests use VirtIO, CPU pinning is sensible, memory is not overcommitted and storage/network queues are not saturated. Most guest work then runs directly on the processor, with relatively few exits. Docker performs best for many small, stateless services with low system-call and network contention because containers share the host kernel. Kata performs best when micro-VMs are kept at a reasonable size and workloads have enough CPU and memory to absorb the extra guest-kernel cost.

Typical registration-week case: During registration, web requests, database queries and login sessions rise together. KVM isolates the application VMs, but the virtual switch, storage queues and hypervisor scheduler can become busy. Docker uses fewer resources, but all containers compete inside the same host kernel and shared storage/network path. Kata provides a stronger boundary, although its guest kernels and virtual I/O add overhead. In all approaches, queueing increases response time before the host reaches 100% CPU utilisation.

Worst case: A noisy research workload can consume CPU, memory bandwidth, storage IOPS and network bandwidth. With memory overcommit, ballooning and swapping cause extra I/O and very high latency. Live migration during peak load also consumes network bandwidth and CPU. KVM and Kata have VM scheduling and virtual-device overhead; Docker containers can interfere through shared kernel and device resources. Throughput falls and login/database latency rises. This is why SIS should have reserved capacity and should not share an unrestricted resource pool with batch jobs.

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM side-channel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries
- The likely impact on the confidentiality, integrity and availability of student records

ANSWER: The main assets are student records, examination results, login credentials and service availability. The trust boundaries are the KVM hypervisor boundary, the shared host-kernel boundary in Docker, the KVM-plus-guest-kernel boundary in Kata, and the management/image/network control planes.

VM escape or hypervisor/runtime vulnerability: A flaw in QEMU, KVM or a device model could allow a guest to affect the host or another VM. KVM and Kata both depend on the hypervisor, although Kata adds a VM boundary around the container. Docker has no separate guest kernel; a container escape through a host-kernel vulnerability can affect the host and other containers. Impact: unauthorised access or modification of student records and possible service outage.

Cross-tenant side channel: Tenants sharing CPU caches or simultaneous multithreading resources may infer information through timing or cache behaviour. All three approaches can

be exposed when placed on the same physical host. VM boundaries reduce direct access but do not remove shared-hardware side channels. Impact: possible leakage of sensitive information.

Unpatched guests and VM sprawl: Old guest kernels and forgotten VM images increase the chance of exploitation. This is especially relevant to KVM and Kata because each VM/guest kernel needs patching. Docker reduces the number of kernels but still requires host-kernel and image patching. Impact: compromise, persistence or denial of service.

Image tampering and insecure migration: Freely copied images may contain malware, old packages or secrets. An attacker who alters an image can introduce malicious code. Unencrypted or unauthenticated migration traffic may expose memory contents or allow tampering. These risks affect VM images in KVM/Kata and container images in Docker. Impact: loss of confidentiality and integrity.

Virtual-network sniffing or spoofing: A poorly configured virtual switch or bridge may allow one tenant to observe or impersonate another tenant's traffic. Docker bridges and hypervisor virtual networks both require segmentation and access controls. Impact: stolen credentials, altered traffic or denial of service. SIS data must be protected through network micro-segmentation, encryption and strict access policies.

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)
- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

ANSWER: Let xK, xD and xA represent the selected fractions of KVM, Docker and Kata instances; let n be instances per host; let d be the number of dedicated SIS hosts; and let z

indicate whether SMT is enabled. Let P be measured performance degradation, I be the isolation score, and Cost be infrastructure and operating cost.

Objective and decision rule: Minimise $J = \alpha(\text{Cost}) + \beta(P) + \gamma(1/I)$, where α, β and γ are weights chosen by the university. A high γ gives greater importance to protecting student records. The objective discourages both excessive cost and weak isolation while accounting for performance.

Constraints: The design must satisfy $P \leq 0.10$ for SIS during registration peak; SIS and external research tenants must have separate trust boundaries; usable memory per host must not exceed 480 GB under the stated assumption; CPU allocation must not exceed the chosen overcommit policy; and the total number of hosts used must be no more than 16. SIS hosts should be dedicated or strongly isolated from external tenants. The final decision is to use KVM for SIS/LMS and separate Kata or dedicated KVM pools for external research, rather than placing external tenants in ordinary shared-kernel containers on SIS hosts.

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

ANSWER – mitigation 1: dedicated host pools: Place SIS and examination VMs on dedicated hosts, separate from external research jobs. This reduces noisy-neighbour risk and limits cross-tenant exposure. Residual risk remains from management-plane compromise, insider access and shared storage/network systems. The cost is lower density because reserved hosts cannot be filled by other tenants. With 16 hosts, reserving 4 for SIS leaves 12 for other workloads; the exact capacity must be validated against peak demand. This trade-off is acceptable for critical student records.

Mitigation 2: patching and signed images: Maintain supported guest kernels, scan images, sign approved images and allow deployment only from a controlled registry. This reduces exploitation of outdated guests and image tampering. Residual risk remains from zero-day vulnerabilities and compromised signing credentials. Image scanning and signature verification add small deployment-time overhead, not a major per-request cost. This is acceptable for SIS.

Mitigation 3: secure migration and network micro-segmentation: Use authenticated, encrypted live migration; isolate management traffic; apply per-tenant virtual networks, firewall rules and least-privilege access. This reduces interception, spoofing and lateral movement. Residual risk remains if an administrator account or endpoint is compromised. Encryption consumes some CPU and network capacity, so migration should be scheduled outside registration peaks. The small performance cost is acceptable.

Mitigation 4: disable SMT for sensitive workloads: Disabling SMT on SIS hosts reduces some shared-core side-channel exposure. It may reduce available logical CPU scheduling capacity, so the effective capacity model should use physical cores only and avoid counting sibling threads as full cores. Residual side channels through shared caches and other hardware remain. The density cost is acceptable for SIS if the host pool is sized with sufficient physical cores.

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately **1 → 2 → 4 → 8 → 16 co-located instances** (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memory-intensive test.
- **Interference test:** run at least one noisy-neighbour experiment.

- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

ANSWER – reproducible test plan: The following is a practical test procedure. It is not a claim that the experiments have already been run. Use the same laptop or test server, instance CPU/memory limits, workload duration and number of repetitions for each approach. If nested virtualisation is unavailable, record that limitation and reduce the instance counts.

Test platforms: KVM: Ubuntu Server with KVM/QEMU and VirtIO. Docker: Docker Engine with CPU and memory limits. Kata: Kata Containers with a KVM-based runtime. Record CPU model, RAM, kernel version, hypervisor/runtime versions and whether nested virtualisation is enabled.

Scale and workloads: Test 1, 2, 4, 8 and 16 instances if hardware permits. Run a CPU test with sysbench or stress-ng, disk testing with fio, network testing with iperf3, and a memory test that allocates a controlled amount of RAM. Repeat each test at least three times.

Example commands: CPU: sysbench cpu --threads=2 --time=60 run. Disk: fio --name=educloud --rw=randrw --bs=4k --size=1G --runtime=60 --time_based. Network: iperf3 -c SERVER_IP -t 60. Record the exact commands and configuration used. Run a separate idle baseline on bare metal.

Noisy-neighbour and security checks: For the interference test, run a CPU or disk stress workload in one instance while measuring response time in another instance. For Docker, inspect namespaces and cgroups using docker inspect and the host's namespace/cgroup tools. For KVM/Kata, inspect the running VM process and confirm that workloads have separate guest kernels. Record screenshots or command output as evidence.

TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance

- Performance degradation versus a bare-metal baseline
- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

ANSWER – recording method: No measured values are supplied in the assignment file, so actual results must be entered after running Task 8. Do not present predicted values as experimental measurements. For every approach and instance count, record at least three repetitions and calculate mean and standard deviation.

Results table to complete: Record: approach; number of instances; CPU score/throughput; disk IOPS or MB/s; network throughput; mean latency; 95th-percentile latency; CPU steal time; startup time; memory per instance; and performance degradation against bare metal.

Calculations: Mean = (sum of the repeated measurements)/number of repetitions. Standard deviation describes variation between repetitions. Performance degradation (%) = ((bare-metal throughput − virtualised throughput)/bare-metal throughput) × 100. For latency, report both mean and 95th percentile because occasional slow requests matter during registration.

Graphs to include: Graph 1: number of instances on the horizontal axis and throughput on the vertical axis, with separate lines for KVM, Docker and Kata. Graph 2: number of instances or test condition on the horizontal axis and 95th-percentile latency on the vertical axis, showing normal and noisy-neighbour runs. Add error bars using the standard deviation.

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5

Interpret the results; do not simply reproduce tables.

ANSWER – comparison method: After completing Task 9, compare the measured results with the Task 3 predictions. The model predicts that per-instance CPU capacity falls as the overcommit ratio rises, memory capacity reaches its 60-instance theoretical ceiling under the stated assumptions, and network share falls approximately as 22.5/N Gbps when N instances share the link.

Interpretation: If Docker gives the highest throughput at low instance counts, that is consistent with its lower virtualisation overhead. If KVM has slightly lower density but more stable isolation, that is consistent with the separate guest-kernel design. If Kata has higher startup time or memory use, that is expected from its micro-VM layer. These are expected trends, not claimed test findings.

Error analysis: Calculate percentage error as ((measured value − predicted value)/predicted value) × 100. Differences may result from CPU cache effects, scheduler behaviour, storage caching, network packet size, nested virtualisation, background host processes or the fact that the model assumes equal sharing. Explain the largest difference using the test configuration and raw measurements.

Security conclusion: Performance measurements alone cannot prove security. Use the Task 8 checks to confirm kernel sharing, namespace/cgroup boundaries, VM separation and mitigation status. If an unexpected result shows that isolation or patching is not configured as intended, revise the threat assessment and recommendation.

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs

- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

ANSWER: I recommend a hybrid architecture. Use KVM/QEMU with VirtIO for SIS, LMS and the examination portal. Place the most sensitive SIS and examination VMs in a dedicated host pool with reserved CPU and memory. Use ordinary Docker containers for trusted, stateless internal services where high density and fast deployment are important. Use Kata Containers or a separate KVM pool for external research collaborators.

Reason and performance: KVM provides a mature VM boundary and supports different guest operating systems. VirtIO reduces device-emulation overhead. Docker is more resource-efficient for suitable internal services, while Kata gives external workloads a stronger boundary than shared-kernel containers. The model gives a 60-instance memory ceiling per host under the stated 8 GB demand and 32 GB reserve; actual SIS density must be lower if peak CPU, storage or network measurements require it.

Security and operations: The design separates sensitive tenants, uses signed and scanned images, timely guest/host patching, encrypted migration, micro-segmented networks and central monitoring. Residual risks include hypervisor vulnerabilities, side channels, administrator compromise and shared storage/network failures. Operations require a maintained image catalogue, patch windows, access reviews, capacity monitoring and tested backup/recovery.

Scalability and limitations: As EduCloud grows beyond 400 instances, host pools can be expanded and workloads placed according to sensitivity and resource demand. The main limitations are the cost of dedicated capacity, extra Kata memory, and the need to manage several runtime types. Capacity should be expanded based on measured peak demand rather than simply increasing overcommit.

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

A. IMPACT ANALYSIS – ANSWER: The new Spark/Hadoop workloads are long-running and use heavy disk and network I/O. Registration-week SIS traffic is about three times normal, so the previous assumption of a mostly trusted and moderate workload mix is no longer valid. External collaborators are less-trusted tenants, and regulations require demonstrable separation of student records. Batch jobs can create noisy-neighbour effects by filling storage queues, network links and CPU resources. The threat model now includes deliberate or accidental cross-tenant access, not only faults between university departments.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

B. EVALUATION OF THE ORIGINAL DESIGN – ANSWER: A shared pool is no longer adequate for SIS and external research. Even if the 22.5 Gbps usable network estimate is shared fairly, the per-tenant share falls as active workloads increase, and real batch traffic

may be uneven. At three times normal registration load, the SIS needs reserved CPU, memory, storage IOPS and network capacity. Ordinary Docker containers alone are not a sufficient boundary for external tenants because they share the host kernel. The original hybrid design is a useful starting point but needs stronger host-pool and network separation.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

C. ADAPTATION OR REDESIGN – ANSWER: I would MODIFY the design. Keep SIS and examination workloads on dedicated KVM hosts with reserved resources. Keep LMS on a separate KVM pool or an appropriately isolated internal pool. Place external Spark/Hadoop clusters on a separate research host pool using KVM VMs or Kata micro-VMs. Give the research pool dedicated NICs or SR-IOV virtual functions where supported, and use separate storage policies and virtual networks. Encrypt and authenticate live migration, restrict it to a management network, and schedule maintenance migration outside registration peaks. This reduces cross-tenant attack surface and noisy-neighbour interference, but lowers average density because capacity is reserved.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisy-neighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

D. EVIDENCE-BASED DEFENCE – ANSWER: The revised design follows from the model: the host has 480 GB of assumed usable RAM, so 8 GB per instance gives a theoretical maximum of 60 before other limits; CPU and I/O can impose a lower safe density. With three-times registration demand and long-running research jobs, shared resource fairness alone cannot guarantee SIS latency. Dedicated host pools provide predictable capacity and a clear isolation boundary. KVM/Kata adds overhead compared with ordinary containers, but this is justified for student data and external tenants. The final deployment should be accepted

only after the Task 8–10 experiments confirm the required latency and throughput targets, including noisy-neighbour and migration tests.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6
Task 4 – Performance Under Operating Conditions	4
Task 5 – Security Threat and Vulnerability Analysis	3
Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4
Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**